



汉字自动拆分论文

汉字拆分自动化综述

据笔者了解，在长期实践过程中前人提出了两种有价值的通往自动拆分实践，分别称为「嵌套拆分」和「笔画表达式」。

嵌套拆分

这一方法的来源已不可考。在较为有名的方案中，徐码是基于这一方法的。

GB 字集的汉字 90% 以上都是形声字、会意字等合体字，这些字可以自然划分成两个小一些的字，而小字又能划分成更小的字，最终得到 700 左右基本部件。对于形码中的「拆字」，大多是针对这 700 基本部件而言的，很少有跨过自然分部界限的拆分。

当我们完成 700 基本部件的拆分之后，结合适当的部件组合的结构信息，理论上我们可以确定剩下的 6000 字的拆分结果；因而可以大大减少工作量。但是：

1. 仍然需要手工拆分很多字，遇到改拆分规则等情况，一样无能为力；
2. 不能保证自然分部的正确性，保不齐就会有一些合乎规则的拆分跨过分部界限，而作者通常意识不到这一点，造成编码错误。

笔画表达式

这一方法由法月（百度贴吧用户名：逆卷炎灵）在输入法吧的讨论中首次提出。

汉字是写出来的，是一笔一画写的，所以汉字无非就是**笔画种类信息**和**笔画之间位置关系信息**的堆砌。如果我们找到一种方法把这些信息完整地记录下来作为一个包含若干对象的数据库，而拆字就是字对象和字根对象之间的运算，而拆分规则可以通过描述字根对象的若干特征来引入。

笔者曾成功实现过这种方法，并意识到它存在一定的优势：

- 如果要增加字根，只需要添加一个字根对象，然后会自动根据运算规则给出新的拆分结果；
- 改动拆分规则时，只需要改动拆分规则算法，而不需要改动数据库。

但是，在实际应用中，笔者发现：

1. 将复杂的、准连续的汉字图形信息量化时，很难做到不损失信息。如果损失了信息，很可能导致人能够区分开的不同汉字计算机区分不开。

2. 因此，很难控制好表达式中到底要加入多少信息。比如，两个笔画在何处相交，在正中间相交还是偏左偏右，这个信息要不要记录？不记录，有些情况分不开；记录，有些情况又分得太开，不符合习惯.....
3. 最重要的是，判断什么样的字或字根需要区分本来是因人而异的事情。有的作者可能觉得「上横比下横长」就不叫字根「二」了，有人觉得还叫「二」.....但是如果用了笔画表达式，相当于把表达式编写者一个人的判断标准强加给了所有人。

吸取笔画表达式的失败经验，我们可以得出结论：**如果要实现自动拆分，其核心数据库必然是非量化的。**

直线与直线

设直线 $\mathbf{a}(t)$ 和 $\mathbf{b}(t)$ 是以 $\mathbf{A}_0, \mathbf{A}_1$ 以及 $\mathbf{B}_0, \mathbf{B}_1$ 确定的直线，那么它们相交的条件可以表示为：

$$(1 - t_1)\mathbf{A}_0 + t_1\mathbf{A}_1 = (1 - t_2)\mathbf{B}_0 + t_2\mathbf{B}_1$$

这等价于一个非齐次线性方程：

$$(\mathbf{A}_1 - \mathbf{A}_0, \mathbf{B}_0 - \mathbf{B}_1) \begin{pmatrix} t_1 \\ t_2 \end{pmatrix} = \mathbf{B}_0 - \mathbf{A}_0$$

它的解可由 Cram 规则确定。分以下几种情况：

1. $\delta < t_1 < 1 - \delta, \delta < t_2 < 1 - \delta$: 相交
2. $|t_1| < \delta$ 或 $|t_1 - 1| < \delta$ 或 ...: 相连（很多种）
3. 其余情况：相离（可根据投影进一步区分）

直线与曲线

设直线 $\mathbf{a}(t)$ 和曲线 $\mathbf{c}(t)$ ，它们相交的条件可以表示为：

$$(1 - t_1)\mathbf{A}_0 + t_1\mathbf{A}_1 = (1 - t_3)^2\mathbf{C}_0 + 2t_3(1 - t_3)\mathbf{C}_1 + t_3^2\mathbf{C}_2$$

两方程加减消去 t_1 ，得到关于 t_3 的二次方程，得到两解；再代回方程，得到两个对应的 t_1 的解。两对解分别得到一组关系（对于复数的情况，绝对值变为取模），然后按优先级确定最终的关系。

曲线与曲线

设曲线 $\mathbf{c}(t)$ 和 $\mathbf{d}(t)$ ，它们相交的条件可以表示为：

$$(1 - t_3)^2 \mathbf{C}_0 + 2t_3(1 - t_3) \mathbf{C}_1 + t_3^2 \mathbf{C}_2 = (1 - t_4)^2 \mathbf{D}_0 + 2t_4(1 - t_4) \mathbf{D}_1 + t_4^2 \mathbf{D}_2$$

两方程加减消去 t_3^2 ，然后解出 t_3 代入另一式得到一元四次方程。实际问题表明该方程在 $[-\delta, 1 + \delta]$ 上无解或有一解，用二分法数值求解即可。

TrueType 字体构成

TrueType 字体由多段一次或三次 Bezier 曲线构成：

- 一次曲线由起点、终点构成；
- 三次曲线由起点、终点、控制点 1 和控制点 2 构成；

一次曲线（即直线）就是连接起点和终点的直线，而三次曲线是一条参数曲线并由下式给出：

$$\mathbf{b}_3(t) = \mathbf{P}_1(1 - t)^3 + \mathbf{P}_2 3t(1 - t)^2 + \mathbf{P}_3 3t^2(1 - t) + \mathbf{P}_4 t^3$$

坐标系

- x 轴正方向为右方，格点取值为 0 至 1000；
- y 轴正方向为上方，格点取值为 -100 至 900；

命令

移动类

- `hmove(x)`：横向移动 `x` 个格点；
- `vmove(y)`：纵向移动 `y` 个格点；
- `rmove(x, y)`：横向移动 `x` 个格点，然后纵向 `y` 个格点；

直线类

- `hline(x1, [y1, ...])`：横向绘制长 `x` 的线段；其余可选参数按 `y1, x2, y2, x3, ...` 交替出现，即然后纵向 `y1` 的线段，依此类推；
- `vline(y1, [x1, ...])`：纵向绘制长 `y` 的线段，其余与 `hline()` 类似；
- `rline(x1, y1, [x2, y2, ...])`：绘制直线段使其横向位移为 `x1`，纵向位移为 `y1`，其余类似；

曲线类

- `hvcurve(x1, y1, x2, y2)`：横向移动 `x1` 个格点到达第一个控制点，然后纵向移动 `y1`、横向移动 `x2` 达到第二个控制点，最后纵向移动 `y2` 到达终点，由此确定了一条 Bezier

曲线；

- `vhcurve(y1, x1, y2, x2)`：类似；
- `hhcurve(x1, x2, y1, x3)`：横向移动 `x1` 个格点到达第一个控制点，然后横向移动 `x2`、纵向移动 `y1` 达到第二个控制点，最后横向移动 `x3` 到达终点，由此确定了一条 Bezier 曲线；
- `vvcurve(y1, y2, x1, x3)`：类似；
- `rrcurve(x1, y1, x2, y2, x3, y3, [x4, y4, ...])`：移动 `(x1, y1)` 到达第一个控制点，移动 `(x2, y2)` 到达第二个控制点，移动 `(x3, y3)` 到达终点，然后循环往复；

复合类

- `rlinecurve(x1, y1, x2, y2, x3, y3, [x4, y4, ...], x0, y0)`：相当于 `rrcurve(x1, y1, x2, y2, x3, y3, [x4, y4, ...])` 和 `rline(x0, y0)` 的复合；
- `rcurveline(x0, y0, x1, y1, x2, y2, x3, y3, [x4, y4, ...])`：相当于 `rline(x0, y0)` 和 `rrcurve(x1, y1, x2, y2, x3, y3, [x4, y4, ...])` 的复合。